O(|V|) operations to update priority or to adjust the queue after extraction.

With the remaining two choices, the running time is O(|V|*log(|V|) + |E|*DQ(|V|)) where:

- |V| is the number of vertices on the graph;
- |E| the number of edges;
- DQ(|V|) is the (average) running time of each "priority update" operation.

Table 14.1 Running time of Dijkstra's algorithm, on a connected graph with |V| vertices and |E| edges.

	Array	Heap	Fibonacci Heap
Running Time	$O(\|V\|^2)$	O(|E|*log(|V|))	$O(\|V\|*\log(\|V\|) + \|E\|)^{\S}$

(§) Amortized.

Table 14.1 summarizes the running-time of Dijkstra's algorithm, relating it to the implementation of priority queue used.

The best theoretical result is obtained with Fibonacci heap, for which the amortized time to decrease the priority of an element is O(1). However, this data structure is complicate to implement and inefficient in practice, so our best bet is using heaps. As we have seen in section 2.9, d-way heaps allow us to have a more efficient implementation, in practice.